

SMART — improving customer service

C J Selley, S Baker and R McKay

The quality and consistency of customer service offered by telecommunications providers is now a key differentiator in the residential market-place. BT has introduced the 'SMART' customer handling system to provide competitive advantage through world-class levels of customer service. A large-scale, three-tier client/server system, SMART exploits the existing functionality, data and security mechanisms of an established mainframe system and adds ease-of-use, reduced error rates and a focus on 'service' and 'selling'. The development and deployment experiences of the SMART programme provide lessons in the delivery of large-scale distributed systems in a business-critical environment. The key issues of security, performance, resilience and operability are discussed and the solutions implemented for SMART are presented.

1. The business challenge

BT faces ever-increasing levels of competition in the UK residential telephony market-place. Cable companies, long-distance service companies and mobile operators all offer competing products to entice customers from a market-place of 22 million people.

Marketing techniques are becoming more sophisticated. Free call allowances, progressive discounts for higher call usage, discounts for calls within calling circles and free or discounted network-based services are all part of the marketing game. Faced with a barrage of operators all touting ostensibly similar products with a confusion of tariffs and options to pick from, customers will often place their business with the supplier who offers the best overall quality of service — a fair price, reliable service, understandable bills, a helpful and efficient enquiries service. 'Customer service' is becoming a critical differentiator in the residential market-place. Current support systems development is aimed at offering a customer experience of BT on the major inbound sales channel ('150') which is second to none.

The '150' sales channel is a formidable undertaking. BT has 34 customer service centres across the UK, employing 5000 customer service agents (CSAs), taking five million customer calls each month. Customers call to place orders for telephone service, select services, charging options (such as Friends and Family) or to make enquiries about bills, payment methods (such as direct debit or budget account) and BT's entire range of products, services and tariffs. This wide range of call types imposes major challenges for the channel. The agents must understand the full product set and be able to advise on the most appropriate services for individual customers. They must be

fully conversant with billing and payment policy. They must be adept at spotting sales opportunities and articulate in conveying benefits in the course of a short telephone conversation. They must be competent at processing orders and enquiries against the Customer Service System (CSS) — the powerful computer support system which supports a wealth of order- and bill-related functionality. They must also be efficient — with such vast call volumes to cater for, fast and accurate call handling is essential to controlling the costs of the channel.

CSS provides a broad range of business functions, developed and honed over many years. It is supported on a number of large mainframe computers and has a partitioned database. Customer and account data is held in a single logical database and distributed on a geographic basis across 29 physical databases. The combination of broad functionality and vast data supports the day-to-day operations of the '150' channel agents. Through CSS, an agent may examine and update customer, account and order records for any one of the 22 million customers. Through a technique known as 'transaction switching', transparent access is provided from each of the 34 customer service centres to each of the 29 databases.

Although immensely powerful, CSS does pose some challenges for the '150' channel. The CSS training period for CSAs is six weeks. The familiarisation period, to full competence with CSS for all call types is estimated at six months. The CSS system is efficient for order taking and enquiry taking, but is not oriented towards 'selling'. The need to invoke a sequence of CSS programs to fulfil a single business function (e.g. provision of telephony service may require 20 programs) can lead to user error and in the worst

case can impact upon customer service. Hence the need for SMART — a system that will provide a new interface to CSS services and the services of other ‘host’ systems to achieve:

- reduced call handling times, to reduce costs,
- reduced system error rates, to improve the quality of customer service,
- reduced system training times, to reduce costs,
- improved selling support, to improve the quality of customer service and enhance ‘150’ channel revenues.

2. SMART — a distributed system

2.1 Design overview

SMART is a 3-tier distributed system (see Fig 1). At the desktop tier, a PC runs the Microsoft Windows NT/3.51 operating system. The NT operating system supports multi-threaded applications — fundamental to the design of the SMART application. Multi-threading is used to achieve customer profiling against the CSS database without adversely affecting call-handling times. It is also used to process orders and ‘customer contact’ records against CSS on behalf of the CSA while the CSA moves on to talk to the next customer.

The principal middle-tier components of SMART are:

- standing data database,
- order error data database,
- products and service data database,
- Connection Manager,
- System Monitor/6000,
- SNA server.

All but the SNA server are supported on a replicated Unix platform. The SNA server is supported on a replicated NT/AS platform and serves to provide a 3270 service to CSS. This is required at log-on time (when SMART screen-scrapes a native log-on) and on-demand from the CSA, to make use of an emulator (Attachmate Extra) for CSS access in native mode for business functions outside of the current scope of the SMART product.

The desktop SMART application provides a graphical interface to the user, offering a set of 25 basic ‘business services’ such as ‘provide telephone service’, ‘provide product’, ‘provide chargecard’, ‘bill display’ and ‘provide charging option’ via graphical icons. The application enforces business processes, many of which are driven from rules held in the mid-tier standing data database. Data entry

is via simple forms, radio buttons or simple menu selections. Data vetting on input ensures that the CSA is offering appropriate products for the customer. Checks are made to ensure that services offered are compatible with exchange technology and with the customer’s other existing services.

The desktop application interacts with the Oracle-based products and services database (PSD) and with the CSS hosts via a component known as System Manager. This presents a data model view of the basic business entities (customer, product, service, bill, etc) and its role is to mask the population of those data entities from either the PSD or CSS. In practice, interactions with the PSD map to SQL*net/ODBC calls to the Oracle mid-tier database and interactions with CSS map to DCE/RPC request messages sent via the mid-tier Connection Manager component. The application programmers are concerned with the data map and not with the source or sink of the data (PSD or CSS).

Connection Manager provides a communications service to CSS via a BT common Middleware component, SISS (systems interconnect service shell). Connection Manager supports a simple message set which is available to the desktop application — ‘logon’, ‘transfer’ and ‘logoff’. SISS provides a simple interface for transparent message passing to CSS and has the role of providing protocol conversion between TCP/IP and LU6.2. In addition, it provides a routing function, passing messages to the appropriate CSS host. SISS itself is provided in two distinct components, an ‘API’ component which is linked to Connection Manager and a ‘server’ component, which receives traffic from multiple Connection Managers on multiple mid-tier servers located at the customer service centres, and provides the protocol conversion and routing function. Figure 2 illustrates the system’s physical design.

The generalised messaging interface (GMI), another BT common Middleware component, services ‘logon’ and ‘logoff’ requests from clients (SMART application instances) received via Connection Manager and SISS. Secure tokens are exchanged between the application instance and GMI for each GMI ‘logon’. An application instance may request up to three concurrent ‘logon’ sessions, in order to support three concurrent CSS interactions on behalf of the user. This technique is fundamental to the delivery of ‘reduced call-handling times’ when measured against a CSS user operating a 3270 terminal in native mode. Every client application-invoked ‘transfer’ request must be accompanied by the relevant secure token.

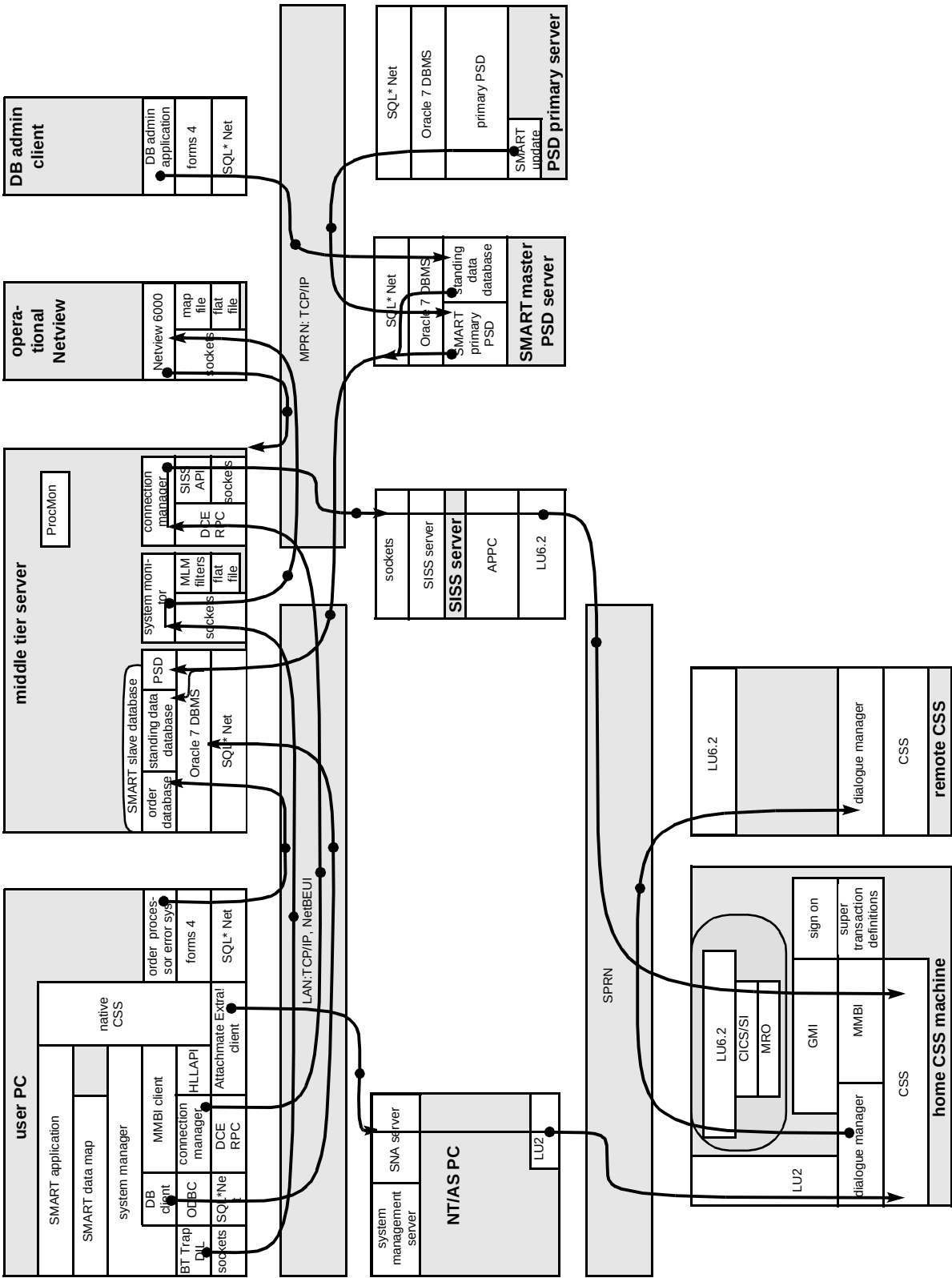


Fig 1 Block diagram of SMART.

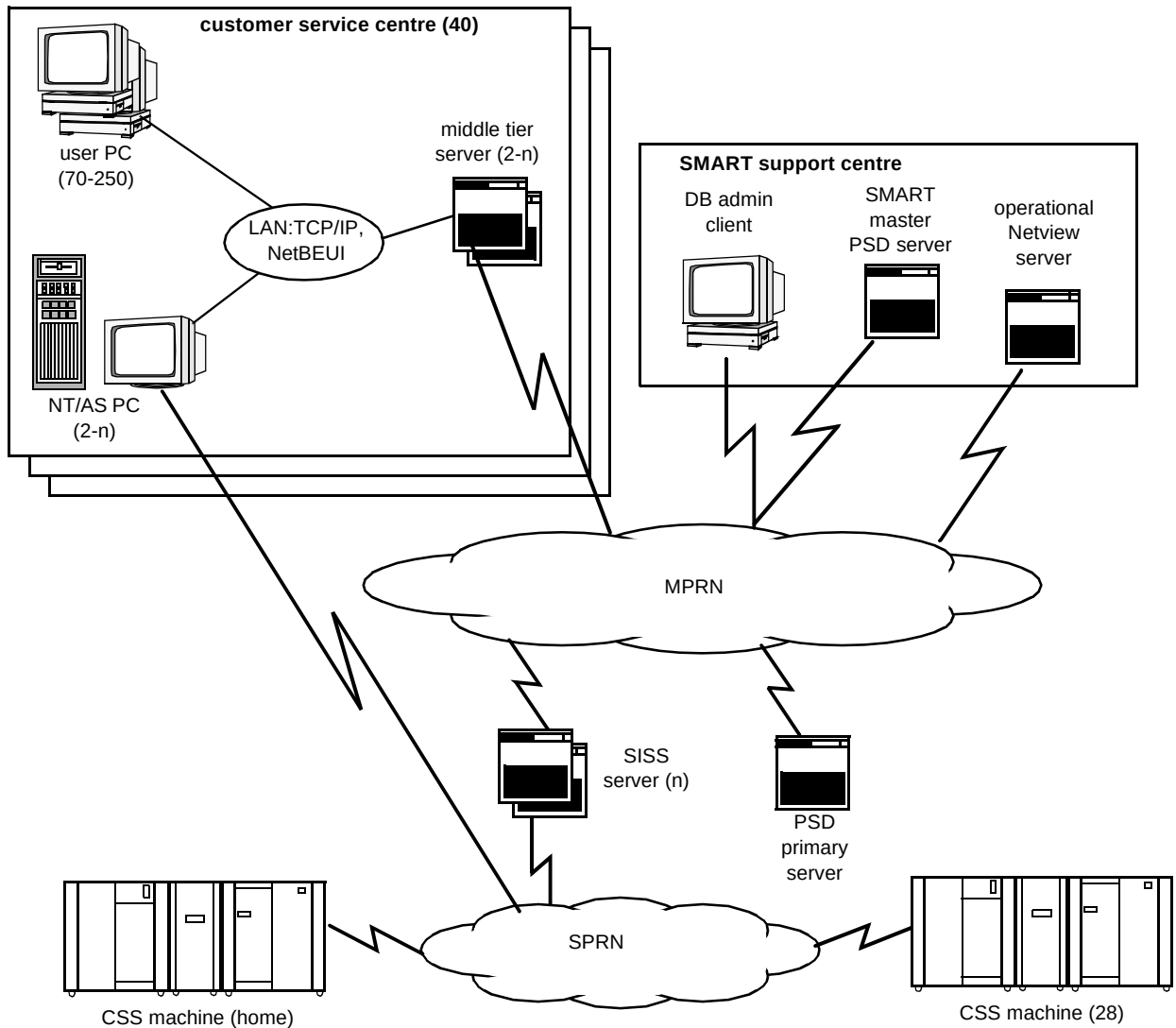


Fig 2 SMART physical design.

The products and services database is maintained at a single site and is subject to daily updates. The Oracle 'snapshot' facility is used to mirror changes to the PSD master to each of the read-only copies held on local mid-tier servers. Local storage on a mid-tier server on the same local area network (LAN) underpins fast response times for all PSD accesses.

2.2 Faster than CSS

In concept, the requirement to work faster than CSS was difficult to meet. If SMART is considered to be merely a graphical user interface on to CSS, then order placement and enquiry servicing would be executed at the same speed as CSS used in native mode. The SMART design offers a simple technique for addressing the speed requirement. It is based on an observation of the call pattern which is common to the majority of order call types on the channel.

The call commences with a salutation from the CSA. During the second phase of the call, the customer explains the nature of the enquiry and the CSA services the enquiry. This may involve the execution of one or more 'read' transactions on CSS — perhaps to examine the customer's bill, perhaps to examine the customer's installation records. The call typically closes (third phase) with the use of update transactions to process an order and to register an issue (or customer contact record) recording the customer interaction. An 'issue' is raised as a means of recording brief details on every customer interaction with BT and is regarded as a prerequisite for world-class customer service. Figure 3 shows a typical call profile, while Fig 4 illustrates a SMART call profile with reduced call-handling time.

The SMART design supports this third phase of the call, the order processing/issue processing phase, without CSA or customer involvement. SMART captures the order processing/issue processing requirement during the second

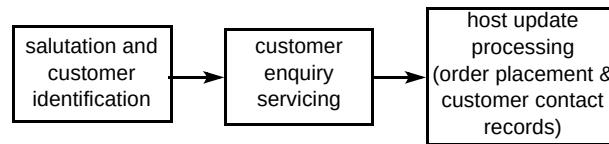


Fig 3 Typical call profile.

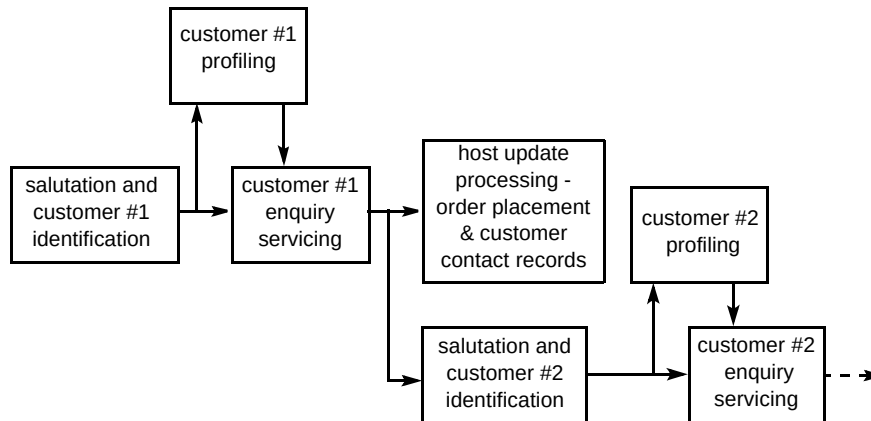


Fig 4 SMART call profile for reduced call handling time.

phase of the call and automates the processing against the host system. During the third phase of the call, the customer is not involved and the CSA is free to deal with the next call on the channel. By this technique 'call-handling time' is reduced. The technique applies to approximately 80% of '150' channel calls and is not applicable in instances of orders where customer interaction is necessary, e.g. appointment making or selection of a telephone number. Concurrent processing within SMART is supported by concurrent threads both in the application and in Connection Manager, and by the mechanism of multiple concurrent logical channels into CSS supported by GMI.

To guard against host (CSS) update failures, which would not be noticed by the CSAs since they run in a background thread, SMART stores all updates on the local mid-tier server database ahead of processing against CSS. In the event of a successful CSS update the record is immediately deleted from the mid-tier database. In the event of a failure, the record persists and a separate, off-line Oracle forms application is supplied to allow the order database to be managed by a pair of users at each customer service centre.

SMART was designed as a Windows application, to be driven primarily from the mouse, augmented by keyboard input. Practical experience during the early months of roll-out has provided some useful feedback. Users find switching between mouse and keyboard to be a hindrance. It adds seconds to call-handling times, which may seem petty,

but when considered in the context of call-handling time targets in the region of 260 seconds, it can be significant. It is likely that future SMART developments will support keyboard-only operation in addition to mouse-and-keyboard operation.

Windows applications do not support the type-ahead features of the block-mode character-based interfaces. In certain scenarios, CSAs using native CSS are to be found typing into screens which have not yet appeared! Their familiarity with certain programs on CSS is such that they simply ignore CSS response times and type on regardless.

2.3 As available as CSS

The CSS has evolved over a decade or more. The development, implementation and support processes have been refined and honed and the 'system' achieves an impressive level of availability for a system of such scale. It services a population of some 60,000 on-line users and supports batch programs producing tens of thousands of customer bills per day.

For the defined 'on-line' day and for defined performance levels (expressed in terms of screen response times), CSS achieves availability levels in excess of 99%. The introduction of SMART is intended to have no adverse impact on CSS availability as perceived by the CSAs. Hence, when CSS is available, SMART must be available.

This poses a challenge, since SMART is implemented in a three-tier structure with PCs at the desktop, Unix and NT/AS at the middle tier and CSS at the host tier. The SMART solution offers more components, each a potential point of failure. The approach to maximising SMART availability is based on the following principles.

- Over-provision of the desktop device — to cater for the possibility of PC failures, each CSC is over-provided by 5%. In practice it is more economical to over-provide standard desktop PCs than to provide highly-reliable PCs or to pay the cost of a very fast PC support contract.
- Replication at the middle tier — the Unix and NT/AS mid-tier servers are replicated. Each user has an Oracle account (with replicated password) at two Unix mid-tier servers at the same centre. Each user may pick up a terminal session via either one of two mid-tier NT/AS servers. Recovery from loss of a Unix mid-tier server is not transparent. The log-on process supports the concept of primary and secondary mid-tier servers and detects non-availability of the primary and initiates recovery to the secondary. Mid-tier server failure necessitates a logoff and logon by the affected users.
- Each user has a principal CSS account and a secondary CSS account. The two accounts are on different hosts. The secondary account will be used only in the event that the CSS host serving the primary account is unavailable. While use of the secondary account, under these circumstances, still precludes access to customer or network data held on the 'unavailable' CSS, all data on other CSSs will be accessible and hence most '150' calls will be serviced without an interruption to service.
- The key 'network' component in the SMART architecture is the systems interconnect service shell (SISS) server. Loss of (or slow response from) a particular SISS server is detected at run time and the SMART mid-tier server will redirect traffic via an alternative SISS server without loss of service to the user.
- The underlying 'bearer' network supporting traffic between SMART and CSS is common to that which currently supports traffic between terminals (block-mode, IBM 3270) and CSS.
- Under failure conditions which render the Connection Manager-SISS-CSS path unavailable, and when the Attachmate-SNA server-CSS path is available, then recourse to native CSS provides a fallback of last resort.

In the production environment SMART matches CSS availability.

2.4 Easier to train than CSS

CSS supports thousands of programs. Each is invoked by a program name — a 3 or 4 letter acronym (e.g. MFFN, 'modify Friends & Family Numbers'). PCD CSAs are conversant with approximately 200 programs which they use on a regular basis to service the range of call types on the '150' channel.

SMART 'hides' the CSS programs from the CSA. The SMART desktop application has a Windows 'look-and-feel'. Core business functions are presented as icons on an icon bar. Once invoked, the business function will lead the agent through a sequence of inputs and outputs — some of which will involve dialogue with the customer. The application is structured around business functions which have meaning to the customer and have been derived from a requirements capture technique based on Jacobsen's use-case method [1]. SMART supports:

- direct debit/budget account,
- itemised billing,
- postpone payments,
- payment advice,
- duplicate invoice,
- charging options,
- select services,
- provide product/service,
- cease product/service,
- shift product/service,
- provide PSTN service,
- cease PSTN service,
- changed number interception,
- renumber,
- bill enquiry,
- sales prompt,
- refunds,
- chargecards,
- order enquiry,
- phonebook ordering,
- directory entry.

CSS provides ~200 commonly used programs. SMART provides ~20 business functions. CSS programs are self-contained and alone do not satisfy a business process.

SMART business functions implement complete business processes for which CSS training is six weeks and SMART training is one week.

2.5 *Less error prone than CSS*

CSS in its current form provides a set of low-level functional building blocks from which business processes can be assembled. Hence, to provide a new customer with basic telephony service might require a CSA to invoke 15 or 20 CSS programs. Some programs would manipulate the new account, perhaps recording the detail of charging options or payment methods. Some would manipulate the installation, recording the apparatus and network-based services required. Some would record details of the customer. Some would be concerned with appointments, others with the allocation of a telephone number. There is no single program to 'provide telephony service'.

The partitioning of CSS functionality into such a large number of small building blocks provides a high degree of flexibility. Business processes may be re-engineered quickly, essentially because the 'process' is captured in the sequence of CSS program invocations and not enforced by the programs themselves. The process is implemented by the CSA and basic process changes are implemented in written instructions to the agents. The drawback to this approach lies in the inconsistency of 5000 agents following written instructions. This leads to errors and can be illustrated by examples.

- Certain network-based services (CallMinder, Call Waiting) are only available to customers served from specific exchange technology. Yet on CSS there is a facility to order network-based services and there are no checks to validate the installation for an appropriate exchange type. The business process step of 'validate exchange technology for chosen network-based service' is a rule which can be serviced via CSS, since CSS offers a program for analysing exchange technology, but its invocation is a process step to be implemented by the agent.
- Current rules for the 'Friends and Family' product permit only a single 'mobile' number and a single 'international' number to be included in the list of five telephone numbers for a particular customer. Yet the CSS programs permit any number of telephone numbers (up to 30) to be included for a customer and no check is made within the program to police the 'one mobile number' limit or the 'one international number' limit. This provides the PCD business with the flexibility to change the features of the Friends and Family (F&F) product, but, in practice, imposes a responsibility on the CSAs to implement the business rules concerning the number mix for the F&F product.

SMART assumes a greater responsibility for the implementation of business processes. It implements product compatibility checks, product-network compatibility checks and the full set of business rules regarding charging products. During the pilot trial, SMART users achieved a significant reduction in error rates on the host system. This has a direct impact on customer satisfaction — there is nothing worse than selling a product to the customer only to discover at a later date that the service cannot be provided.

SMART has delivered a business rules engine implemented at the client and middle tiers of the Architecture. It has avoided the need to enhance the suite of CSS programs to achieve this objective. Flexibility has been accomplished through the definition of simple rules defined in data and held at the middle tier. The most volatile rules may be altered through data-only changes.

2.6 *Support 'selling'*

SMART profiles all customers as soon as they are identified. The agent will enter customer account number or telephone number into the 'welcome' screen and SMART will initiate profiling of the customer to determine basic data such as call bill size, payment history, products purchased/rented, services rented. The customer profile is matched against a set of marketing segments. For each marketing segment, a product set is referenced. A 'sales prompt' is displayed, indicating the most beneficial products/services to be offered to the particular customer. It would, for example, indicate 'Premierline' for a high call bill customer who does not currently have the relevant charging option. It also calculates the potential saving from Premierline based on the latest call bill.

All market segmentation rules and product sets are defined in simple data structures at the mid-tier of the SMART system. They are read once at log-on time and re-used for each customer call.

The sales support features are designed to enable customers to make the most of BT. BT has a broad product set and must talk to its customers at every opportunity about products of relevance to them. This feature is designed to enhance customer service and generate additional revenues.

In order to provide rich and accurate customer profiling during the course of a call, SMART interacts with CSS to read customer-, account- and network-related data. To do so at the beginning of each call would, if carried out in sequence before all relevant enquiry/order handling, extend the call-handling time. This would be unacceptable. Within the SMART desktop application, customer profiling is carried out within a separate application thread. This allows the agent to service the customer request without delay. Typically the sales prompt appears ~15 seconds into the

call. Despite the elimination of any call-handling time impact, customer profiling does impose an impact at the host. The profiling must be executed for every call (3 million/month) and the nature of the profiling is such that it requires a range of data available from programs within different CSS subsystems (billing, networks, order handling, repair). In practice the task of profiling has given rise to the development of a 'service' transaction on CSS which amalgamates database 'reads' taken from numerous programs in several CSS subsystems into a single new program. The service transaction delivers a much-simplified interface to the SMART application, improved run-time performance and reduced processing load on CSS, as compared with the alternative of making use of the existing CSS programs.

Sales support facilities in the area of charging options are particularly sophisticated. In support of the massive TV-based marketing campaign behind the Friends and Family product, SMART provides a facility to read a customer bill from CSS, analyse the contents with respect to the F&F product and recommend the five best numbers to the customer. The focus throughout is on implementing business processes end-to-end. SMART fills functional gaps on CSS and provides complete business process support.

Segmentation data may be changed by the customer using a separate Oracle forms application supplied with SMART. Forms are supplied to permit users to manipulate many of the business 'rules' that determine the behaviour of the SMART application at run-time.

2.7 As secure as CSS

The security of BT's data is a high-visibility issue inside and outside the company. Security procedures to protect mainframe data have been put in place at some cost, and the introduction of SMART had to maintain the high level of security CSS had already put in place. In addition, the cost of SMART's access control had to be minimised without jeopardising its effectiveness.

The security issues surrounding SMART can be categorised as follows:

- access control to the host data SMART relays to the users,
- access to the storage of customer data on the failed order database,
- access to software on the client PC.

Access to CSS

Existing screen-scraping applications connect to the host systems via a standard 3270 emulator alone, but this

was insufficient for SMART, due to its requirement for three concurrent CSS sessions. Current security arrangements dictate that a user may only connect one session at a time to prevent fraudulent reuse of a known user ID and password.

GMI, a component of common Middleware was enhanced to provide concurrent session access for SMART. Since it cannot be accessed directly by user terminals, the single session restriction was lifted for connections via GMI. However, trebling the number of sessions for each of SMART's 5000 users would have imposed an unacceptable load on CSS resources. As a compromise, each of SMART's sessions on GMI was defined as a cut-down session holding no context information.

When a SMART user connects to CSS using GMI, GMI reuses the existing CSS authentication functions to validate the user ID and password. Assuming success, one GMI token per session is returned to Connection Manager. It in turn creates one user token per session which can be used by the system manager component of the SMART client.

When the client wishes to invoke an MMBI (Middleware message-based interface) transaction on CSS, it supplies one of its user tokens to Connection Manager, which validates the token, and passes the request to CSS with a valid GMI token. GMI, given a valid GMI token, will allow the execution of the transaction and return the result to Connection Manager. This component returns the result to the SMART client, together with a new user token which replaces the original one used.

This constant changing of user tokens between client and Connection Manager means that even if a token is fraudulently intercepted, it can only be used once. Its subsequent (legitimate) use by the real SMART client would be detected and flagged to the operations unit using the BTTrap mechanism.

Using the existing CSS user ID and password validation means that a user needs only to be disabled using the standard CSS procedures to effectively lock them out of SMART as well. The only alternatives to this would be costly and require an inordinate amount of co-ordination.

Access to the mid-tier

Security restrictions need not be so heavy with respect to the PSD. The information contained therein is 'public domain' and need not be so heavily protected. Each user is set up on the mid-tier Oracle servers to be able to access all the PSD data in a read-only capacity. The failed orders are also stored on the mid-tier.

Access to the desktop

Though no key information is stored on the client PCs, it is crucial to ensure that the software installed on the machine is not tampered with in any way. Each user's access is restricted using the Windows NT domain-based security, such that they cannot modify important configurations on the client. Windows NT security also has the benefit of ensuring non-SMART people cannot gain access to the SNA servers, thus maintaining a consistent level of service for the SMART user community.

SMART does not compromise the security of the data stored on CSS, a key requirement for the project. Unfortunately, the simple fact that SMART is based around (relatively) expensive and in-demand PCs makes the likelihood and cost of theft far greater than for 3270 terminals. Despite the security of the data, the security of the hardware cannot be overlooked.

2.8 Low impact on CSS

More than ever before, CSS is changing to respond to market pressures, new products and services and changes in the regulatory climate. As a client of this system, SMART must track changes if it is to continue to operate correctly. Thus, the demands on SMART to accommodate these changes will continue to be high for the foreseeable future. CSS functionality must never be held up by SMART, nor may SMART impact upon the tried and tested 'test, delivery and pilot' processes.

At any one time there are likely to be two major releases of CSS in the field. A user switching between CSS databases may see two completely different versions of the same program, and usually, with a little intuition, can cope with the changes. As intuition is not an option for SMART, it is vital that as soon as a new release of CSS is brought to the pilot stage, SMART is already deployed nationally to deal with any deviations from the previous version of CSS. MMBI protects SMART from cosmetic changes, but where different information is required or available, SMART must operate in a way appropriate to either release. In the most complex case, SMART may only offer a function to the user if the customer they are dealing with is on the latest version of CSS. This adds an extra level of complexity to the configuration management in SMART, and this is one of the greatest challenges for the project.

In order to avoid the necessity of deploying SMART nationally at the same time as a new CSS release enters the pilot stage, SMART has adopted a delivery strategy which mirrors that of CSS. While the 'old' release of CSS is deployed nationally, SMART will deliver a pilot version

which works against both 'old' and 'new' CSS releases. This is a low-risk strategy which enables the operation of a new version of SMART to be monitored closely. Assuming everything is correct, SMART will then be deployed nationally a week or two later. When the 'new' release of CSS enters the pilot stage, all the SMART clients in the field should recognise it correctly and act accordingly. In this way, service-affecting problems can be trapped before the wider user base is affected.

2.9 As operable as CSS

The great strength of a mainframe-based operational support system, in terms of operability, lies in the fact that the mainframe is the principal 'active' device to be managed. Manage it well and a potentially very large user community will benefit from fast performance and high levels of availability. The terminal device is a 'low-tech' item without CD drive, hard drive or floppy disk drive or any of the peripheral items which compromise reliability and pose a serious operability challenge. Yet, when in the three-tier distributed system, the computer at the desktop has a hard drive and access to a mid-tier server with drives, databases and wide-area network connections. There are additional communications devices (in the case of SMART, SISS servers). A three-tier distributed system has all the hallmarks therefore of a less operable, reduced 'availability' option.

SMART offers an approach to four major aspects of operability:

- event and error reporting,
- remote control,
- software distribution,
- desktop platform upgrade.

The key operational support tool employed for SMART is the BT enterprise systems management framework (ESMF), based on IBM's Netview/6000 and System Monitor/6000 products and developed in-house by BT's computing services division.

The production SMART system has been 'integrated' into the framework. The SMART support teams monitor the production installations via a graphical representation of all its constituent hosts and software components on a network of Netview/6000 management servers.

At the system level, ESMF agents (LogMon and ProcMon) monitor the health of mid-tier servers and raise BTTraps under predefined error conditions. At the application level, BTTraps are raised within the code on both the mid-tier (Connection Manager and Oracle) as well as the PC (SMART application and System Manager). The BTTraps raised on the PC are despatched via designated mid-tier servers and all BTTraps are then forwarded to the Netview servers.

For SMART, an effective event-and-error reporting mechanism is a prerequisite for acceptance into production. Without it, service level agreements at customer service centres, which are not permanently staffed with operations personnel, would not be attainable.

Remote control facilities are provided for the use of the SMART support teams. Automated scripts, such as 'start Connection Manager instance', 'stop Connection Manager instance', 'backup PSD master database', are provided to carry out basic remote operations on the mid-tier servers and the primary PSD server. All such scripts are offered as simple menu items via the Netview/6000 proprietary product. The operator utilises a graphical user interface and is not required to log-on to the particular SMART server or

to type native Unix commands. These facilities are made available via IBM's XSA extension to the Netview/6000 product (see Fig 5).

The distribution of the SMART application poses a significant logistical challenge. On a single roll-out date, all 5000 SMART users will expect an upgrade to the application program to be delivered to the PC. These 5000 PCs are located in 34 centres. In practice, software distribution to the desktop is an automatic process. The new application is delivered to Netware servers at each centre. At log-on time, a common control file located on the netware server is read automatically. If the control file indicates the current SMART release to be different to the release on the desktop, the new release is down-loaded automatically. This simple process has proved reliable in operation and straightforward to implement.

An upgrade to the desktop platform is less straightforward. SMART has passed a significant milestone in delivering an automated platform upgrade (operating system and various platform products) to >1000 production desktops, part-way through national roll-out. Two distinct techniques were employed — the first based on a 'gold disk' approach to cloning the base operating system

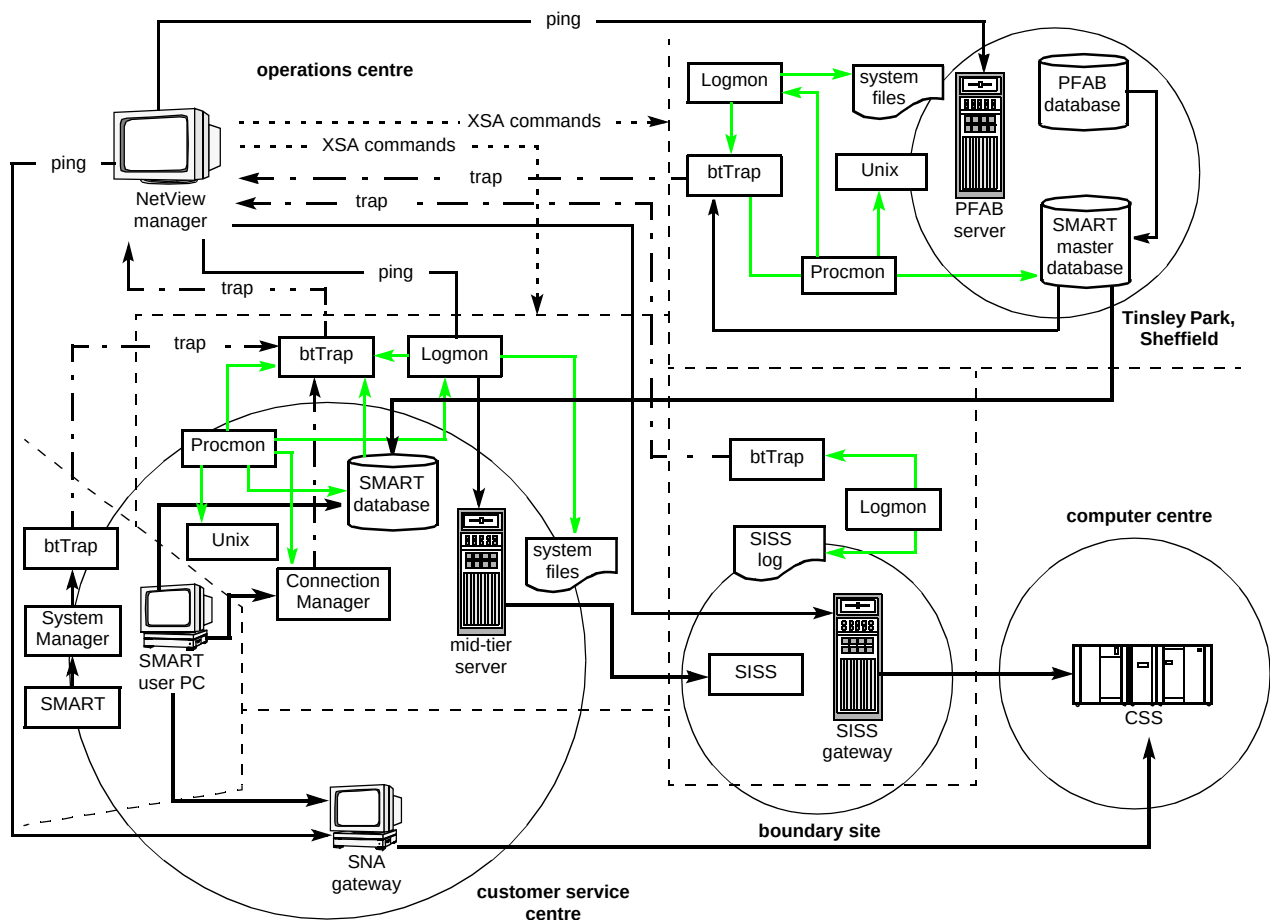


Fig 5 SMART ESMF architecture.

platform, and the second based on the use of a proprietary product, OnDemand's WinINSTALL, which essentially takes 'before' (just operating system) and 'after' (operating system plus full set of platform products) snapshots of a PC in the laboratory and records the differences. WinINSTALL implements the differences automatically on the production platforms with no manual intervention.

A 'gold disk' is constructed from the image of a PC disk in the laboratory with only the operating system installed. The 'gold disk' is downloaded on to the target PC disk, thus giving a fully configured clone of the master. Manual intervention is restricted to a small number of parameters including IP address. The third-party platform products are then loaded. This is achieved using WinINSTALL, which takes 'snapshots' of the PC before and after the manual installation and configuration of one or more software products. The subsequent WinINSTALL package contains instructions covering all the actions that have to be carried out on the PC during installation, e.g. source and destination of files, registry edits, *.INI file edits, icon and group creation, etc. A WinINSTALL package was created for each piece of third-party software and downloaded as a whole on to each target PC as the follow-up step to the 'gold build'. The entire operation took an average 20 minutes to build a SMART PC with only a minimal amount of operator intervention at the outset.

Platform upgrades must be automated — a manual approach is not tenable for production numbers in the thousands. A national deployment of thousands of PCs was implemented over a period of one year. During this time, the hardware vendor changed the configuration of the PC product five times — hence the task of delivering a platform upgrade to the production sites has necessitated six gold disks to cater for the hardware changes imposed by the supplier.

Software distribution to the mid-tier servers has also been subject to partial automation. In full deployment 70 mid-tier servers will be sited at 34 centres. The tool currently in use is Platinum AutoXfer modified and customised by BT to meet its requirements. Packages are created containing compressed archives of the software along with pre- and post-installation command scripts and other configuration data. Currently, connection manager, the Oracle scripts and the ESMF mid-tier configuration files are delivered using Xfer.

3. SMART futures

The technical challenge on SMART is focused on delivering the four key business benefits listed in section 1 in greater measure. To that end, current SMART developments include the following.

- Integration with the customer service centre 'voice platform' — the DMS automatic call distribution (ACD) system which allocates incoming '150' calls to

available CSAs. Computer/telephony integration (CTI) offers the promise of further reductions in call-handling times, through delivery of 'call line identifier' or a telephone number sought from an intelligent voice recognition (IVR) system which answers the customer call ahead of the CSA and solicits the telephone number or account number from the caller. By this mechanism the agent is delivered 'voice' and 'data' simultaneously. Customer profiling can begin the instant that the customer is answered by the CSA. A second benefit concerns the use of CTI technology to allow call-and-data transfer within a customer service centre. This would enable agents to consult managers in-call, to transfer selected calls to appropriately skilled groups of agents and to enable a different approach to training particular groups of agents within the service centre. It opens the possibility of a range of organisation and procedural changes which would deliver improved customer service and potentially lower training costs.

- Integration with the BT operational customer database (OCDB) as a source of customer profiling information — the current approach imposes a significant processing load on the CSS hosts and is subject to onerous real-time performance targets. The OCDB development, scheduled for 1997, will deliver a capability to source much of the profiling data from a database designed for the purpose, offering improved profiling times and reduced CSS loading.
- The SMART product is to be extended to include business functions relating to 'repair'. The BT '151' 'repair' channel will benefit from a simple-to-use graphical interface to fault-record entry, remote line test, fault modification and fault-despatch services, while sharing the more general 'enquiry' and 'ordering' services currently supported in the SMART product.
- SMART will migrate to a more classical three-tier model, utilising CHIS mid-tier services via a DCE/RPC interface, such that business logic (rather than simply the data that drives the business logic) is available from a common middle tier which is shared by a range of applications. CHIS services have proven stability and performance in serving the BT voice applications platform, ACE and the volume business outbound sales system. The promise of CHIS, is to deliver common business services — for SMART that translates to lower development and maintenance costs and lower production costs through the use of shared CHIS platforms.

- SMART will focus on life-style data capture for BT customers. As a sales and marketing tool, life-style data is known to be highly valuable. SMART will offer mechanisms for on-line capture of such data and storage on an appropriate host system. Life-style data will be utilised dynamically during the life of the customer call to tailor the sales prompt displayed to the CSA.

References

- 1 Jacobsen I: 'Object oriented software engineering — a use-case driven approach', Addison-Wesley (1993).



Steve Baker joined BT after gaining an MEng in Software Engineering at Imperial College in 1991. He spent three years designing network management and billing systems based on object database technology.

He now leads a team delivering business-critical systems solutions in the customer handling arena.



Clive Selley joined BT Laboratories, from University in 1986. He worked for 2 years in a development team to specify and implement a communications architecture for the New Generation Systems computing/telephony platform.

In 1988 he joined the BT London Systems Engineering Centre working as an engineer and project manager to deliver a series of network management systems, billing systems and the SMART customer handling system.



Rachel McKay obtained an HND in Computer Studies from Leicester Polytechnic in 1988 and joined GEC-Marconi Software as a software engineer, working on a variety of real-time safety-critical systems for both the civil and defence industry. After graduating from the University of Hertfordshire with a BSc (Hons) in Computer Science in 1992, she joined BT to work in Network Traffic Management. Currently she is the SMART operability team leader within the SMART development project. Her work involves developing, configuring and implementing a wide range of support and software delivery tools for SMART. In addition, she provides consultancy to other BT projects on these areas of open systems operability.